

1. What is inheritance and why it is needed?

Inheritance allows a class to acquire properties/methods of another class, enabling code reusability and hierarchical classification.

Example: `class Car extends Vehicle { }` – Car reuses Vehicle's `start()` method.

2. Types of inheritance with examples

- **Single:** `class B extends A`
- **Multilevel:** `class C extends B extends A`
- **Hierarchical:** `class B extends A`, `class C extends A`
- **Multiple (via interfaces only):** `class D implements X, Y`
- **Hybrid (via interfaces):** combination of above using interfaces.

3. Why Multiple inheritance through classes is not supported in Java?

To avoid the **Diamond Problem** (ambiguity if two parent classes have same method).

Example: If A and B both have `display()`, which does class C extends A, B inherit? Java avoids this by allowing multiple interface inheritance (where methods are abstract).

4. Method Overriding example

When a subclass provides a specific implementation of a parent class method.

java

```
class Animal { void sound() { System.out.println("Some sound"); } }  
class Dog extends Animal {  
    @Override void sound() { System.out.println("Bark"); }  
}
```

5. Runtime vs Compile-time polymorphism

Runtime Polymorphism (Method Overriding)	Compile-time Polymorphism (Method Overloading)
Resolved during program execution	Resolved during compilation
Example: <code>Animal a = new Dog();</code> <code>a.sound();</code> (calls Dog's sound)	Example: <code>add(int a, int b)</code> and <code>add(double a, double b)</code> – chosen at compile-time based on arguments